

Analyse Forensics

Audit avec MVT et Surface d'Attaque

Senghor VLAVONOU

20 avril 2026

Objectif : Réaliser une analyse forensique de smartphones et identifier les indicateurs de compromission (IoC) ainsi que les surfaces d'attaque potentielles.

1 Étape 1 : Audit Pratique avec MVT

1.1 Introduction et Contexte

Dans un contexte où les smartphones constituent aujourd'hui des cibles privilégiées pour des attaques sophistiquées, la détection de logiciels espions avancés représente un enjeu majeur en cybersécurité. Des menaces persistantes avancées (APT), à l'image de Pegasus, sont capables de compromettre un terminal mobile de manière furtive, en exploitant des vulnérabilités système ou des permissions applicatives abusives.

Le présent projet s'inscrit dans une démarche d'analyse forensique visant à évaluer la sécurité de plusieurs smartphones. L'objectif principal est d'identifier d'éventuels indicateurs de compromission (IoC) à travers l'utilisation d'outils spécialisés, tout en analysant les mécanismes pouvant être exploités par un attaquant pour maintenir une persistance sur le système. Cette analyse a été réalisée sur trois appareils distincts : un Infinix Note 12i, un Redmi 13C et un Samsung Galaxy S9.

Dans ce cadre, l'outil Mobile Verification Toolkit (MVT), développé par Amnesty International, a été utilisé afin d'effectuer une extraction et une analyse approfondie des données issues des terminaux audités. Cette approche permet de détecter des traces d'activités suspectes, notamment à travers l'analyse de journaux système, de fichiers générés et d'indicateurs connus de compromission.

Par ailleurs, une cartographie de la surface d'attaque a été réalisée pour chaque appareil, en mettant l'accent sur les permissions critiques accordées aux applications ainsi que sur les zones sensibles du système susceptibles d'être exploitées pour assurer la persistance d'un logiciel malveillant.

Ce rapport présente ainsi les différentes étapes de l'audit, les résultats obtenus ainsi que les analyses associées, dans une optique de compréhension et de renforcement de la sécurité des terminaux mobiles.

1.2 Installation des dépendances

MVT étant écrit en Python, nous avons commencé par vérifier la présence de Python 3 sur la machine.

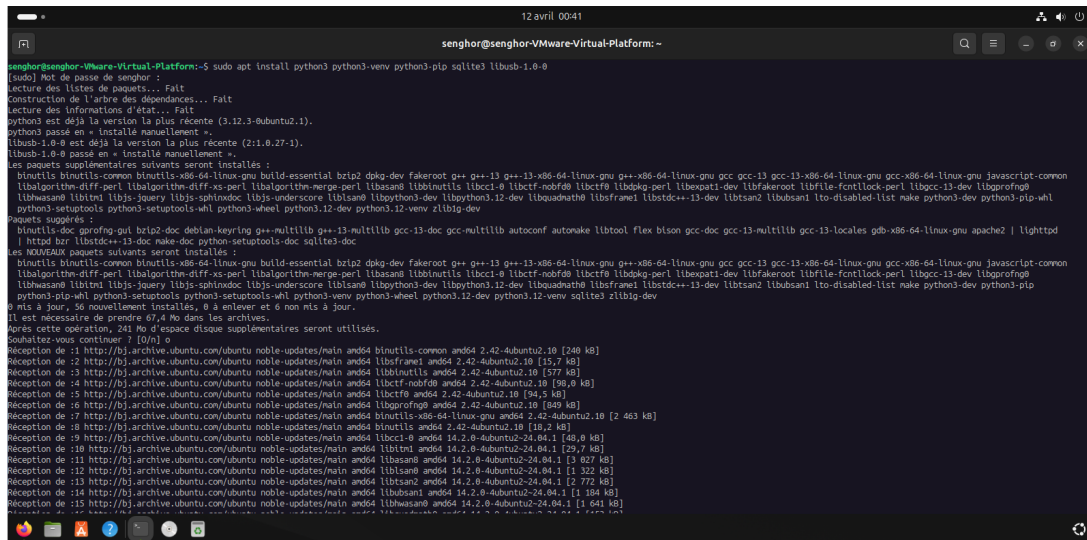


FIGURE 1 – Installation Python 3

Nous avons ensuite installé pipx, l'outil recommandé par MVT pour gérer les installations Python dans des environnements isolés.

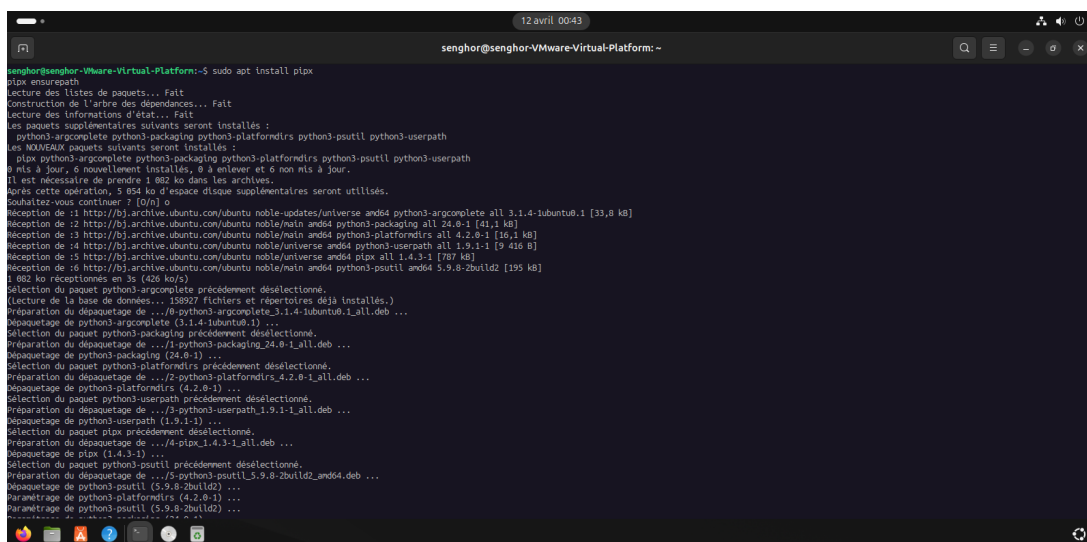


FIGURE 2 – Installation pipx

1.3 Installation de MVT

Avec les dépendances en place, nous avons procédé à l'installation de MVT via pipx.

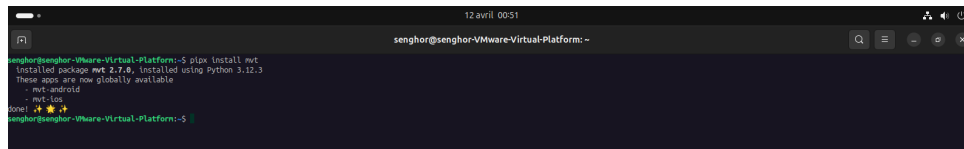


FIGURE 3 – Installation MVT

1.4 Téléchargement des IoCs

Nous avons téléchargé la base d'indicateurs de compromission (IoC) publiée par Amnesty International, nécessaire pour que MVT puisse identifier les menaces connues lors de l'analyse.

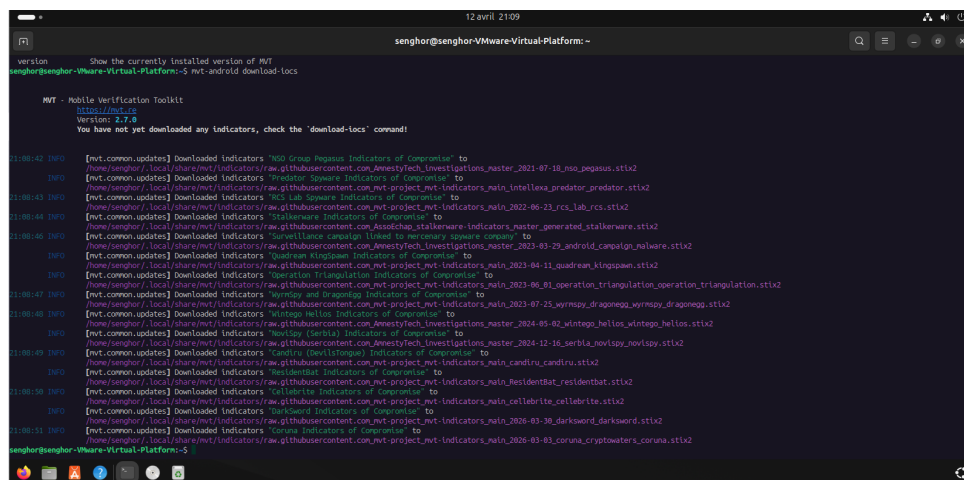


FIGURE 4 – Téléchargement IOCs

1.5 Installation de ADB

Nous avons installé ADB (Android Debug Bridge), l'outil permettant à MVT de communiquer avec les smartphones Android.

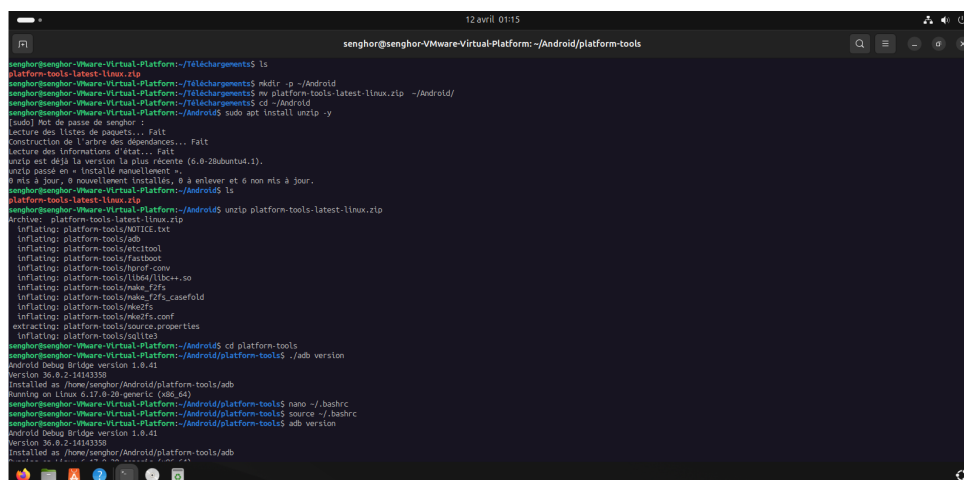
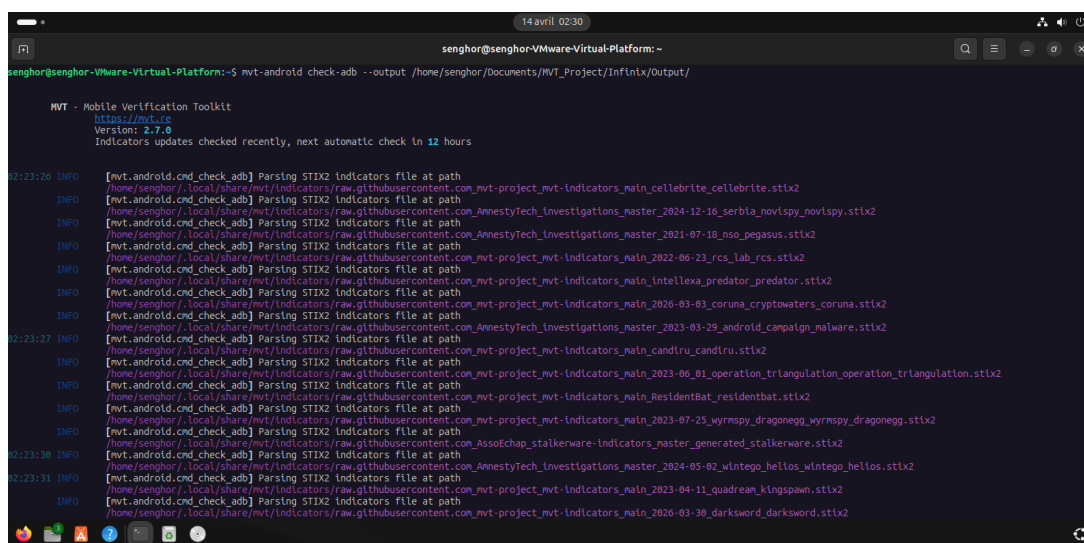


FIGURE 5 – Installation ADB

1.6 Extraction des IoC

La commande `mvt-android check-adb` a été exécutée sur chacun des trois terminaux. La capture ci-dessous, représentative des trois analyses, montre le déroulement de l'opération.



```

senghor@senghor-VMware-Virtual-Platform:~$ mvt-android check-adb --output /home/senghor/Documents/MVT_Project/InfInix/Output/

MVT - Mobile Verification Toolkit
https://mvt.re
Version: 2.7.0
Indicators updates checked recently, next automatic check in 12 hours

[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_cellebrite_cellebrite.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_AnnestyTech_investigations_master_2024-12-16_serbia_novispy_novispy.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_AnnestyTech_investigations_master_2021-07-18_nso_pegasus.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_2022-06-23_rcs_lab_rcs.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_intellexa_predator_predator.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_2026-03-03_coruna_cryptowaters_coruna.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_AnnestyTech_investigations_master_2023-03-29_android_campaign_malware.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_candiru_candiru.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_2023-06-01_operation_triangular_operation_triangular.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_ResidentBat_residentbat.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_2023-07-25_wyrmspy_dragonegg_wyrmspy_dragonegg.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_AsoEchap_stalkerware-indicators_master_generated_stalkerware.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_AnnestyTech_investigations_master_2024-05-02_wlntego_helios_wlntego_helios.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_2023-04-11_quadream_kingspawm.stix2
[INFO] [mvt.android.cnd_check_adb] Parsing STIX2 indicators file at path
/home/senghor/.local/share/mvt/indicators/raw.githubusercontent.com_mvt-project_mvt-indicators_main_2026-03-30_darksword_darksword.stix2

```

FIGURE 6 – Lancement Scan

1.7 Résultats des scans

1.7.1 Infinix Not 12i

Le scan a couvert 303 packages et 247 912 fichiers (32 434 en tmp/media). Les modules Packages, Logcat, RootBinaries et Files ont tous conclu sans détection d'IoC. Cependant, l'analyse a généré un volume important de warnings portant sur la présence de fichiers SUID dans des répertoires non standards, notamment :

- `/proc/sparta/gc_status`, `/proc/sparta/defrag_file`, `/proc/sparta/app_fragment` ces entrées correspondent au sous-système SPARTA, un composant propriétaire MediaTek de gestion de la mémoire flash (NAND/eMMC) présent sur les SoC Helio de ce fabricant.
- `/proc/tran_gc_debug/*` (`gc_time`, `gc_type`, `version`, `drop_cache`, `rom_health`, `emmc_gc_time`, etc.) ces pseudo-fichiers exposent les métriques internes du contrôleur de garbage collection eMMC du chipset MediaTek.

Pourquoi ces warnings? MVT signale tout fichier portant le bit SUID (Set User ID) localisé hors des chemins système standards (`/system/bin`, `/vendor/bin`, etc.). Sur l'Infinix Note 12i équipé d'un SoC MediaTek Helio G85, ces entrées dans `/proc/` sont des interfaces kernel légitimes exposées par les drivers propriétaires du fabricant pour le monitoring du stockage flash. Elles ne constituent pas une menace en elles-mêmes, mais MVT les marque par précaution car leur emplacement sort des conventions AOSP. Ces warnings sont donc des faux positifs liés à l'implémentation spécifique du firmware MediaTek et non des indicateurs de compromission.

1.7.2 Redmi 13C

L'analyse MVT a traité 454 packages installés et indexé 589 252 fichiers au total (dont 144 651 dans les dossiers tmp/media). Les quatre modules exécutés : Packages, Logcat, RootBinaries et Files n'ont produit aucune détection. Aucun indicateur de compromission (IoC) connu n'a été identifié sur ce terminal. Le scan s'est déroulé sans anomalie, ce qui indique que l'appareil ne présente pas de trace de spyware référencé dans la base MVT au moment de l'analyse.

1.7.3 Samsung S9

Le scan a analysé 470 packages et 349 849 fichiers (12 064 en tmp/media). Tous les modules (Packages, Logcat, RootBinaries, Files) sont revenus sans détection. On notera la présence de packages tiers légitimes comme com.snapchat.android, com.instagram.lite, com.vpn.lat ou encore cn.xender, ainsi qu'une entrée ancienne (com.sec.enterprise.knox.shareddevice.keyguard) datée du 31/12/2008, probablement un artefact de date système. Aucun IoC n'a été relevé.

1.8 Fichiers générés

Après l'extraction, MVT génère automatiquement plusieurs fichiers JSON, chacun correspondant à une catégorie d'informations collectées sur le terminal. Voici la liste des différents fichiers JSON collectés.

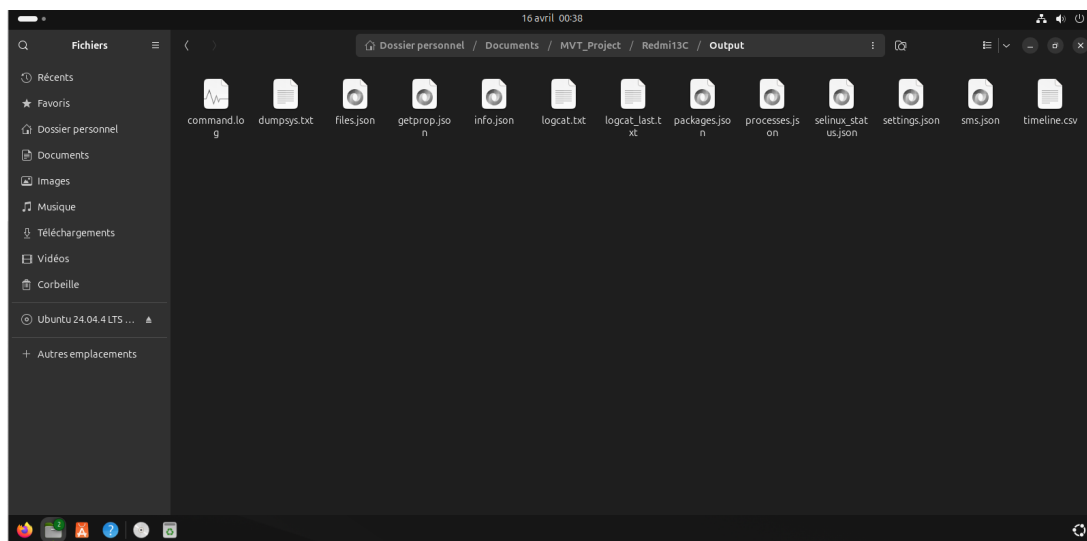


FIGURE 7 – Fichiers Créés

- **files.json** : Inventaire complet du système de fichiers du terminal. Chaque entrée recense le chemin d'accès, la date de modification, les permissions (mode UNIX), la présence de bits SUID/SGID, la taille, le propriétaire et le groupe du fichier.
- **getprop.json** : Liste des propriétés système Android extraites via la commande getprop. Contient les paramètres persistants du noyau, du Bluetooth, de la JVM Dalvik, etc.
- **info.json** : Métadonnées de l'analyse MVT elle-même : version de l'outil (2.7.0), date d'exécution, chemin d'analyse, et liste complète des bases IoC STIX2 chargées lors du scan.

- **packages.json** : Inventaire de toutes les applications installées sur l'appareil, avec leur nom de paquet, chemin APK, installateur d'origine, et statut (activée/désactivée, système ou tierce partie).
- **processes.json** : Snapshot des processus en cours d'exécution au moment de l'extraction : PID, PPID, utilisateur, taille mémoire, état du processus (stat) et label SELinux associé.
- **selinux_status.json** : État du module SELinux (Security-Enhanced Linux) du terminal. Sur l'appareil audité, le mode est enforcing, ce qui signifie que les politiques de contrôle d'accès sont actives et appliquées.
- **settings.json** : Paramètres du système Android exportés depuis la base de données settings.db, incluant les réglages de l'interface (MIUI), la gestion de l'énergie, la mémoire, et les comportements applicatifs.

2 Étape 2 : Cartographie de la Surface d'Attaque

2.1 Infinix Note 12i

2.1.1 Permissions critiques et abusives

L'analyse des 23 applications tierces installées révèle plusieurs permissions sensibles méritant attention. Les deux variantes de WhatsApp (standard et Business) concentrent le plus grand nombre d'accès critiques : microphone, caméra, contacts, stockage externe et comptes utilisateur un cumul qui, bien que justifié par la nature de l'application, constitue une surface d'attaque considérable si l'une d'elles venait à être compromise. Les applications de mobilité Gozem et Yango disposent quant à elles d'un accès permanent à la géolocalisation précise et approximative, une permission dont l'activation continue n'est pas nécessaire en dehors des sessions de trajet actives.

La permission `CHANGE_WIFI_STATE`, accordée à WhatsApp, WhatsApp Business, Boomplay, Yango, Facebook Lite et Xender, est particulièrement préoccupante : elle permet à une application de déconnecter l'appareil d'un réseau Wi-Fi, d'en rejoindre un autre ou d'en modifier la configuration ouvrant la voie à une redirection vers un point d'accès malveillant. Par ailleurs, la quasi-totalité des applications (23/23) dispose de l'accès Internet, et 20 détiennent le `WAKE_LOCK`, leur permettant d'empêcher le processeur de s'endormir et ainsi de maintenir une activité réseau continue en arrière-plan. L'idéal serait d'adopter un modèle de permissions à la demande (runtime permissions) plutôt que des autorisations permanentes accordées à l'installation.

2.1.2 Vecteurs de persistance d'un spyware

Sur ce modèle équipé d'un SoC MediaTek Helio G85, plusieurs points d'ancrage système sont particulièrement exposés. Le processus init (PID 1), racine de l'arborescence des processus, représente le vecteur de persistance le plus puissant : toute modification de ses scripts de démarrage (init.rc) permettrait au spyware de se relancer automatiquement à chaque reboot. Le démon watchdogd, dont le rôle est de surveiller et relancer les processus défaillants, pourrait être détourné pour ressusciter un composant malveillant stoppé. Les threads kernel kthreadd, ksoftirqd et kswapd0 offrent un camouflage efficace par leur caractère permanent et discret, tandis que jbd2/mmcblk0p8, gestionnaire du journal du système de fichiers, pourrait être exploité pour assurer la persistance des modifications au-delà des redémarrages. Enfin, kverityd, responsable de la vérification d'intégrité du système, constitue une cible de choix : son détournement permettrait de neutraliser les mécanismes de détection natifs.

2.2 Redmi 13C

2.2.1 Permissions critiques et abusives

Le Redmi 13C présente la surface d'attaque la plus large des trois terminaux, avec 101 applications tierces dont 4 installées en dehors du Play Store (sideload). L'accès au microphone est partagé par 8 applications incluant Snapchat, WhatsApp, Microsoft Teams, Instagram, Facebook Lite et Shazam. WhatsApp cumule à lui seul cinq permissions critiques simultanées : `READ_PHONE_STATE`, `ACCESS_FINE_LOCATION`, `READ_CONTACTS`, `GET_ACCOUNTS` et `CHANGE_WIFI_STATE` une combinaison qui offre un profil d'identification complet de l'utilisateur, sa position en temps réel, son réseau social et le contrôle de sa connectivité Wi-Fi.

Instagram dispose de la permission `READ_CALL_LOG`, lui donnant accès à l'historique des appels téléphoniques une autorisation sans justification fonctionnelle évidente pour un réseau social. Xender bénéficie d'une géolocalisation précise et approximative sans raison technique convaincante. La permission `CHANGE_WIFI_STATE` est accordée à 28 applications, dont Binance, DeepSeek et AppBlock, ce qui est excessif. Avec 70 applications autorisées à démarrer au boot (`RECEIVE_BOOT_COMPLETED`) et 99 ayant un accès Internet, la capacité de surveillance passive permanente est particulièrement élevée sur ce terminal.

2.2.2 Vecteurs de persistance d'un spyware

L'écosystème MIUI de Xiaomi introduit des vecteurs de persistance spécifiques à ce modèle. `com.miui.daemon` tourne avec les droits `system`, redémarre automatiquement en cas d'arrêt et accède à des fonctions bas niveau du téléphone : un spyware intégré au firmware pourrait s'y loger pour survivre aux redémarrages et aux tentatives de désinstallation. `com.miui.securitycenter.remote`, processus distant du centre de sécurité Xiaomi, consomme une quantité de RAM anormalement élevée (228 Mo) pour son rôle déclaré, dispose des droits `system` et a accès aux permissions, processus actifs et fichiers ce profil en fait un emplacement idéal pour dissimuler une collecte de données. Au niveau kernel, `ueventd` gère les permissions des périphériques `/dev` : son détournement permettrait d'obtenir un accès caché au microphone, à la caméra ou au stockage. Les threads `kworker/R-netns` et `kworker/R-ipv6`, responsables de la gestion de la pile réseau, pourraient maintenir une communication C2 (Command & Control) et relancer automatiquement les connexions d'exfiltration. Enfin, `kauditd`, dont le rôle est précisément d'auditer l'activité système, représente une cible critique : son détournement permettrait à un spyware d'opérer en aveuglant les mécanismes de journalisation natifs.

2.3 Samsung S9

2.3.1 Permissions critiques et abusives

Avec 48 applications tierces, la surface d'attaque du Samsung S9 est nettement plus étendue. WhatsApp, WhatsApp Business, Snapchat et Shazam accèdent au microphone ; WhatsApp, WhatsApp Business, Xender et Snapchat à la caméra. Dix-sept applications disposent d'un accès au stockage externe, incluant Instagram, Pinterest, Facebook, Xender et Snapchat. Facebook et Snapchat peuvent lire l'état du téléphone (`READ_PHONE_STATE`), ce qui donne accès à l'IMEI et aux identifiants réseau des données d'identification pérennes.

Deux permissions ressortent comme particulièrement injustifiées : Duolingo, application d'apprentissage des langues, dispose de l'accès aux contacts (`READ_CONTACTS`), sans que cette permission soit justifiable par ses fonctionnalités déclarées. Xender, outil de transfert de fichiers, détient un accès à la géolocalisation précise et approximative, ce qui va au-delà de son usage nominal. La permission `SYSTEM_ALERT_WINDOW` accordée à Facebook lui permet d'afficher des fenêtres superposées à toutes les autres applications une capacité pouvant être

exploitée pour du phishing par superposition d'interface (overlay attack). La permission `DELETE_PACKAGES` de Xender est également préoccupante car elle autorise la désinstallation silencieuse d'applications. Trente-trois applications reçoivent `RECEIVE_BOOT_COMPLETED`, garantissant leur démarrage automatique au boot, et 17 détiennent `CHANGE_WIFI_STATE`.

2.3.2 Vecteurs de persistance d'un spyware

Le Snapdragon 845 du S9 présente plusieurs processus sensibles. `init` (PID 1) reste le vecteur de persistance ultime. Les nombreux threads `kworker` constituent un terrain de camouflage idéal : leur multiplicité et leur activité générique permettent à un processus malveillant de s'y fondre sans éveiller de soupçon. `ksoftirqd`, gérant les interruptions réseau et I/O, pourrait être exploité pour intercepter le trafic ou maintenir une communication avec un serveur de commande. Les processus `watchdogd` et `msm_watchdog` sont des cibles à double usage : soit détournés pour relancer un malware, soit neutralisés pour empêcher la détection d'un plantage suspect. Les composants GPU (`kgs_l_*`), peu surveillés par les outils de sécurité conventionnels, représentent un vecteur émergent pour le stockage ou l'exécution furtive de code.

3 Conclusion

L'analyse forensique réalisée sur les trois terminaux à l'aide de MVT a permis de vérifier l'absence d'indicateurs de compromission connus, notamment ceux associés à des menaces avancées de type spyware. Aucun IoC issu des bases d'Amnesty International n'a été détecté, ce qui suggère que les appareils audités ne présentent pas de traces d'infection référencée au moment de l'analyse.

Cependant, cette absence de détection ne constitue pas une garantie absolue d'intégrité. L'étude approfondie de la surface d'attaque met en évidence plusieurs faiblesses structurelles, notamment une attribution excessive de permissions sensibles aux applications tierces, ainsi qu'un grand nombre de services capables de s'exécuter en arrière-plan ou au démarrage du système. Ces éléments augmentent significativement le risque d'exploitation en cas de compromission applicative ou système.

Par ailleurs, l'analyse des mécanismes de persistance potentiels révèle que certains composants critiques du système, tels que les processus `init`, `watchdog`, les threads kernel ou encore les services spécifiques aux constructeurs (MIUI, MediaTek, Qualcomm), constituent des points d'ancrage privilégiés pour un attaquant avancé. Leur niveau de privilège élevé et leur exécution permanente en font des cibles idéales pour l'implantation de code malveillant furtif et résilient.

Ce travail met ainsi en évidence que la sécurité d'un terminal mobile ne repose pas uniquement sur la détection d'IoC, mais également sur une gestion rigoureuse des permissions, une surveillance des processus critiques et une compréhension fine de l'architecture système. L'approche combinant analyse forensique et cartographie de la surface d'attaque apparaît donc essentielle pour anticiper et contrer les menaces persistantes avancées.

Enfin, l'utilisation de MVT s'est révélée pertinente dans un contexte d'investigation, mais gagnerait à être complétée par des mécanismes de détection comportementale et de surveillance en temps réel afin d'améliorer la capacité de détection face à des attaques inconnues ou en constante évolution.